

Q1. Write a menu driven program with 4 options (Push, Pop, Display, and Exit) to demonstrate the working of stacks using arrays.

Code:

```
#include <iostream>
using namespace std;

#define MAX 5

int stack[MAX];
int top = -1;

void push()
{
    int x;

    if (top == MAX - 1)
    {
        cout << "Stack Overflow" << endl;
    }
    else
    {
        cout << "Enter value: ";
        cin >> x;

        top++;
        stack[top] = x;

        cout << "Value pushed successfully" << endl;
    }
}

void pop()
{
    if (top == -1)
    {
        cout << "Stack Underflow" << endl;
    }
    else
    {
        cout << "Popped value: " << stack[top] << endl;
        top--;
    }
}
```

```

void display()
{
    if (top == -1)
    {
        cout << "Stack is empty" << endl;
    }
    else
    {
        cout << "Stack elements are:" << endl;

        for (int i = top; i >= 0; i--)
        {
            cout << stack[i] << endl;
        }
    }
}

```

```

int main()
{
    int choice;

    while (true)
    {
        cout << "\n1. Push" << endl;
        cout << "2. Pop" << endl;
        cout << "3. Display" << endl;
        cout << "4. Exit" << endl;

        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            push();
        }
        else if (choice == 2)
        {
            pop();
        }
        else if (choice == 3)
        {
            display();
        }
    }
}

```

```

        else if (choice == 4)
        {
            cout << "Program ended" << endl;
            break;
        }
        else
        {
            cout << "Invalid choice" << endl;
        }
    }

    return 0;
}

```

Output:

```

Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1

```

```

Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter your choice: 1
Enter value: 4
Value pushed successfully

1. Push
2. Pop

```

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
```

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

```
4. Exit
Enter your choice: 2
Popped value: 4
```

```
1. Push
2. Pop
```

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack is empty
```

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
```

```
Terminal
Code + - [ ] [ ] ...

PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
2. Pop
3. Display
4. Exit
Enter your choice: 4
Program ended
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> 
```

Q2. Write a menu driven program with 4 options (Push, Pop, Display, and Exit) to demonstrate the working of stacks using linked-list.

Code:

```
#include <iostream>
using namespace std;
```

```
struct Node
{
    int data;
    Node *next;
};
```

```
Node *top = NULL;
```

```
void push()
{
    int x;

    Node *newNode = new Node;

    cout << "Enter value: ";
    cin >> x;

    newNode->data = x;
    newNode->next = top;
    top = newNode;

    cout << "Value pushed successfully" << endl;
}
```

```
void pop()
```

```

{
    Node *temp;

    if (top == NULL)
    {
        cout << "Stack Underflow" << endl;
    }
    else
    {
        temp = top;

        cout << "Popped value: " << top->data << endl;

        top = top->next;

        delete temp;
    }
}

```

```

void display()
{
    Node *temp;

    if (top == NULL)
    {
        cout << "Stack is empty" << endl;
    }
    else
    {
        temp = top;

        cout << "Stack elements are:" << endl;

        while (temp != NULL)
        {
            cout << temp->data << endl;
            temp = temp->next;
        }
    }
}

```

```

int main()
{
    int choice;

```

```

while (true)
{
    cout << "\n1. Push" << endl;
    cout << "2. Pop" << endl;
    cout << "3. Display" << endl;
    cout << "4. Exit" << endl;

    cout << "Enter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
        push();
    }
    else if (choice == 2)
    {
        pop();
    }
    else if (choice == 3)
    {
        display();
    }
    else if (choice == 4)
    {
        cout << "Program ended" << endl;
        break;
    }
    else
    {
        cout << "Invalid choice" << endl;
    }
}

return 0;
}

```

Output:

```
Terminal Code + - [ ] [ ] ...
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
```

```
Terminal Code + - [ ] [ ] ...
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
4. Exit
Enter your choice: 1
Enter value: 7
Value pushed successfully

1. Push
```

```
Terminal Code + - [ ] [ ] ...
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
```

```
Terminal Code + - [ ] [ ] ...
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
3. Display
4. Exit
Enter your choice: 2
Popped value: 7

1. Push
```

▼ Terminal

Code + View + Trash + ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 3

▼ Terminal

 **Code**     ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

4. EXLL

Enter your choice: 3

Stack is empty

1. Push
2. Pop

▼ Terminal

 **Code**     

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 4

▼ Terminal

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

2. Pop
3. Display
4. Exit

Enter your choice: 4

Program ended

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>
```

Q3. Write a program to convert infix expression into postfix expression using stack.

Code:

```
#include <iostream>
#include <cctype>
using namespace std;
```

```
char stack[100];
int top = -1;
```

```
void push(char x)
{
    top++;
    stack[top] = x;
}
```

```
char pop()
{
    char x = stack[top];
    top--;
    return x;
}
```

```

}

int priority(char x)
{
    if (x == '^')
        return 3;

    if (x == '*' || x == '/')
        return 2;

    if (x == '+' || x == '-')
        return 1;

    return 0;
}

int main()
{
    char infix[100];
    char postfix[100];

    int i = 0;
    int j = 0;

    cout << "Enter infix expression: ";
    cin >> infix;

    while (infix[i] != '\0')
    {
        if (isalnum(infix[i]))
        {
            postfix[j] = infix[i];
            j++;
        }
        else if (infix[i] == '(')
        {
            push(infix[i]);
        }
        else if (infix[i] == ')')
        {
            while (top != -1 && stack[top] != '(')
            {
                postfix[j] = pop();
                j++;
            }
        }
    }
}

```

```

    }

    pop();
}
else
{
    while (top != -1 && priority(stack[top]) >= priority(infix[i]))
    {
        postfix[j] = pop();
        j++;
    }

    push(infix[i]);
}

i++;
}

while (top != -1)
{
    postfix[j] = pop();
    j++;
}

postfix[j] = '\0';

cout << "Postfix expression: " << postfix << endl;

return 0;
}

```

Output:

```

Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter infix expression: A+B*C
Postfix expression: ABC*+
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> 

```

Q4. Write a program to convert infix expression into prefix expression using stack.

Code:

```
#include <iostream>
#include <cstring>
#include <cctype>
using namespace std;

char stack[100];
int top = -1;

void push(char x)
{
    top++;
    stack[top] = x;
}

char pop()
{
    char x = stack[top];
    top--;
    return x;
}

int priority(char x)
{
    if (x == '^')
        return 3;

    if (x == '*' || x == '/')
        return 2;

    if (x == '+' || x == '-')
        return 1;

    return 0;
}

int main()
{
    char infix[100];
    char postfix[100];
    char prefix[100];
```

```

int i, j;
int n;

cout << "Enter infix expression: ";
cin >> infix;

n = strlen(infix);

for (i = 0; i < n / 2; i++)
{
    char temp = infix[i];
    infix[i] = infix[n - i - 1];
    infix[n - i - 1] = temp;
}

for (i = 0; i < n; i++)
{
    if (infix[i] == '(')
        infix[i] = ')';
    else if (infix[i] == ')')
        infix[i] = '(';
}

i = 0;
j = 0;

while (infix[i] != '\0')
{
    if (isalnum(infix[i]))
    {
        postfix[j] = infix[i];
        j++;
    }
    else if (infix[i] == '(')
    {
        push(infix[i]);
    }
    else if (infix[i] == ')')
    {
        while (top != -1 && stack[top] != '(')
        {
            postfix[j] = pop();
            j++;
        }
    }
}

```

```

        pop();
    }
    else
    {
        while (top != -1 && priority(stack[top]) > priority(infix[i]))
        {
            postfix[j] = pop();
            j++;
        }

        push(infix[i]);
    }

    i++;
}

while (top != -1)
{
    postfix[j] = pop();
    j++;
}

postfix[j] = '\0';

n = strlen(postfix);

for (i = 0; i < n; i++)
{
    prefix[i] = postfix[n - i - 1];
}

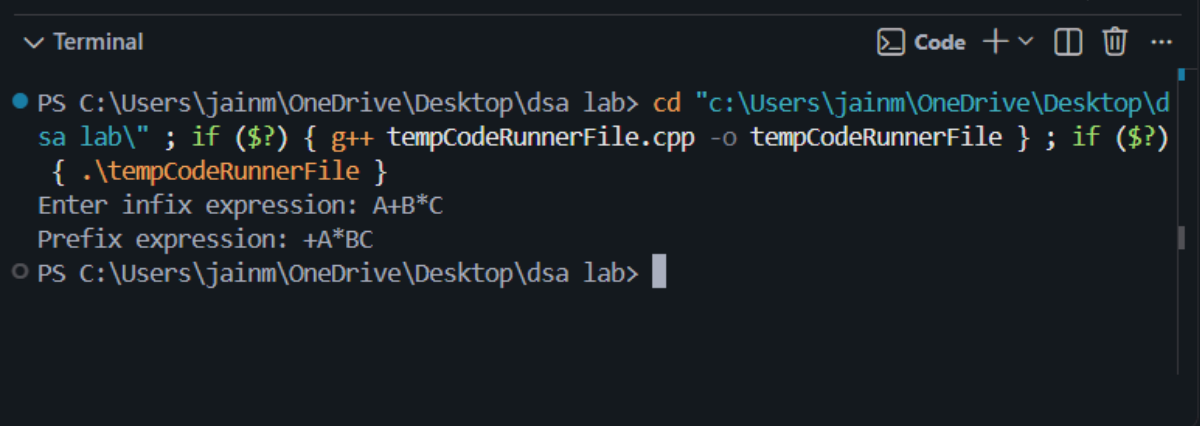
prefix[n] = '\0';

cout << "Prefix expression: " << prefix << endl;

return 0;
}

```

Output:



```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter infix expression: A+B*C
Prefix expression: +A*BC
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>
```

Q5. Write a program to evaluate a postfix expression using stack.

Code:

```
#include <iostream>
#include <cctype>
using namespace std;
```

```
int stack[100];
int top = -1;
```

```
void push(int x)
{
    top++;
    stack[top] = x;
}
```

```
int pop()
{
    int x = stack[top];
    top--;
    return x;
}
```

```
int main()
{
    char postfix[100];

    int i;
    int a, b, result;

    cout << "Enter postfix expression: ";
    cin >> postfix;
```

```
i = 0;

while (postfix[i] != '\0')
{
    if (isdigit(postfix[i]))
    {
        push(postfix[i] - '0');
    }
    else
    {
        b = pop();
        a = pop();

        if (postfix[i] == '+')
            result = a + b;
        else if (postfix[i] == '-')
            result = a - b;
        else if (postfix[i] == '*')
            result = a * b;
        else if (postfix[i] == '/')
            result = a / b;

        push(result);
    }

    i++;
}

cout << "Result = " << pop() << endl;

return 0;
}
```

Output:



```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter postfix expression: 23+
Result = 5
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>
```

Ln 62 Col 2 Spaces: 4 LTF: 8 CRLF: C++ (v) Go Live